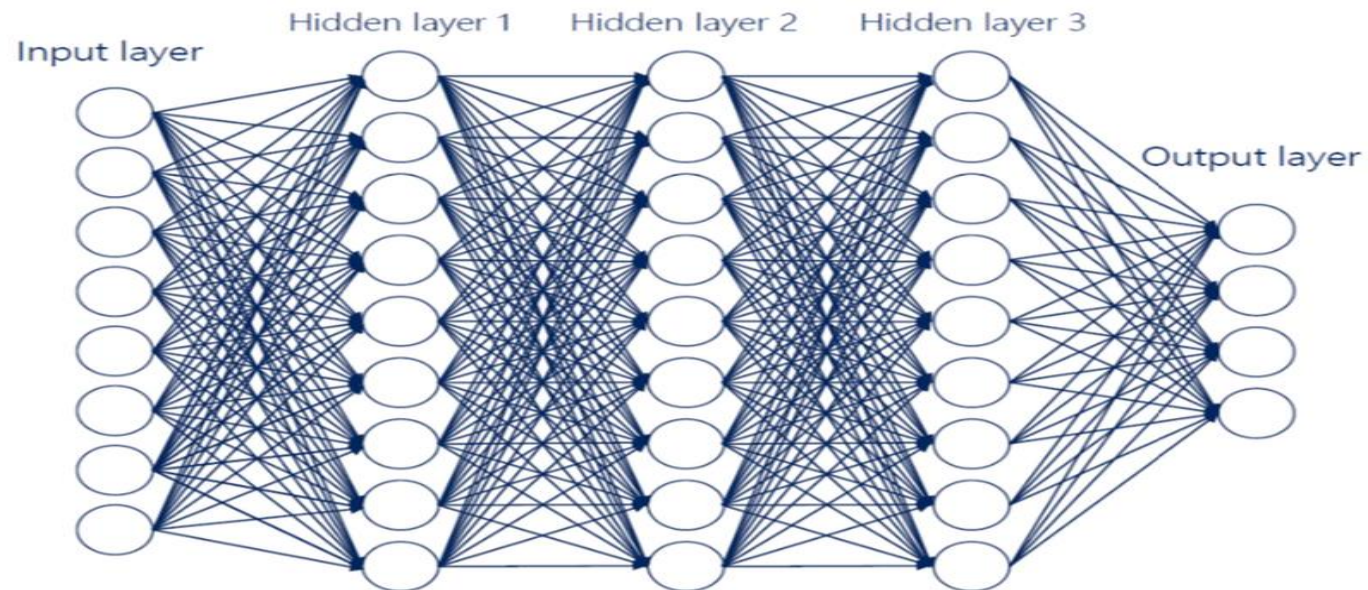
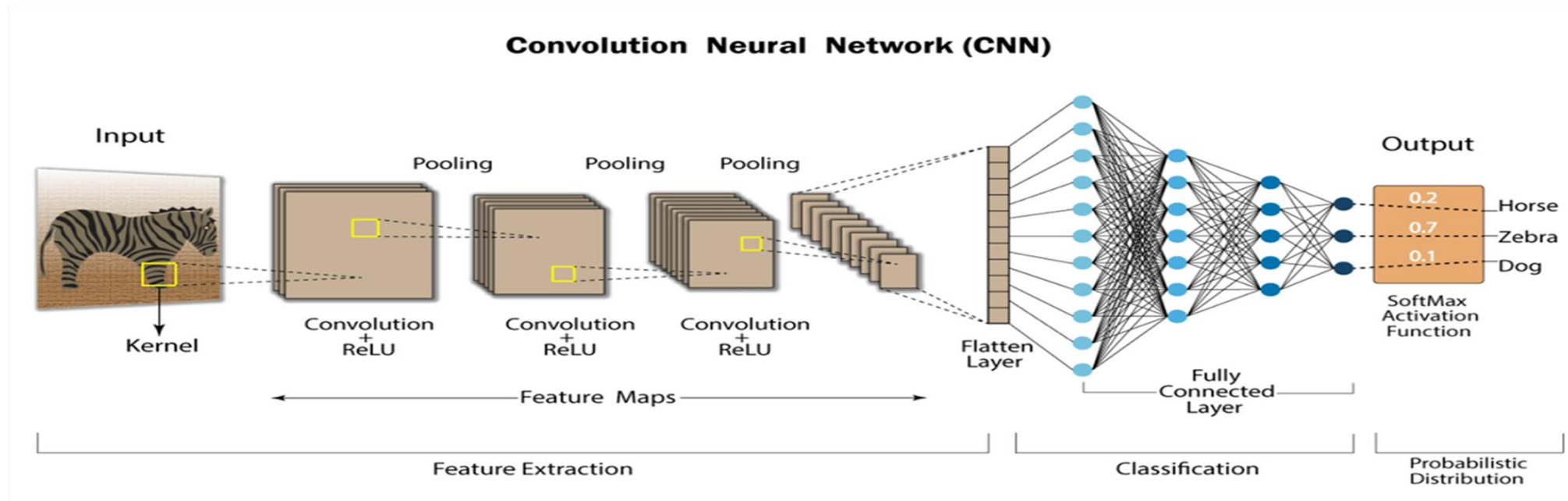


DL lec 10

Autoencoder

In supervised learning



Autoencoder

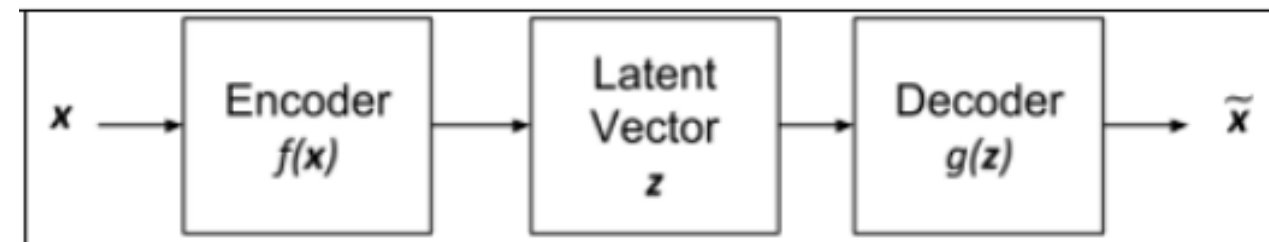
- Autoencoder is an **unsupervised artificial neural network** that **learns** how to efficiently compress and encode data then learns how to reconstruct the data back from the **reduced encoded representation** to a **representation that is as close to the original input** as possible.

- In its simplest form, an autoencoder will **learn** the representation by trying to **copy** the **input** to **output**.
- An autoencoder will **encode the input distribution into a low-dimensional tensor**, which usually takes the form of a vector. This will approximate the hidden structure that is commonly referred to as the **latent representation**, code, or vector.
- The latent vector will then be **decoded** by the **decoder** part to **recover the original input**.
- As a result of the **latent** vector being a low-dimensional compressed representation of the input distribution, it should be expected that the output recovered by the decoder can only approximate the input.

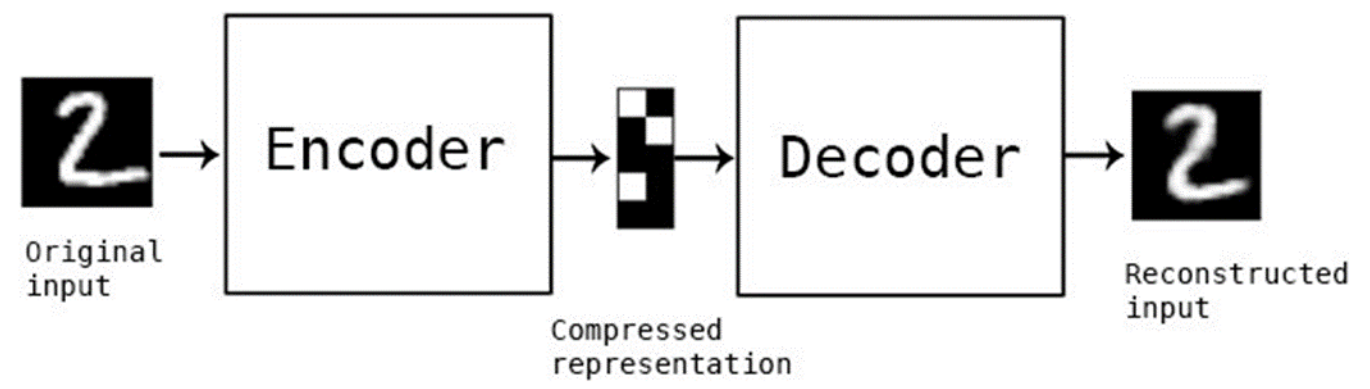
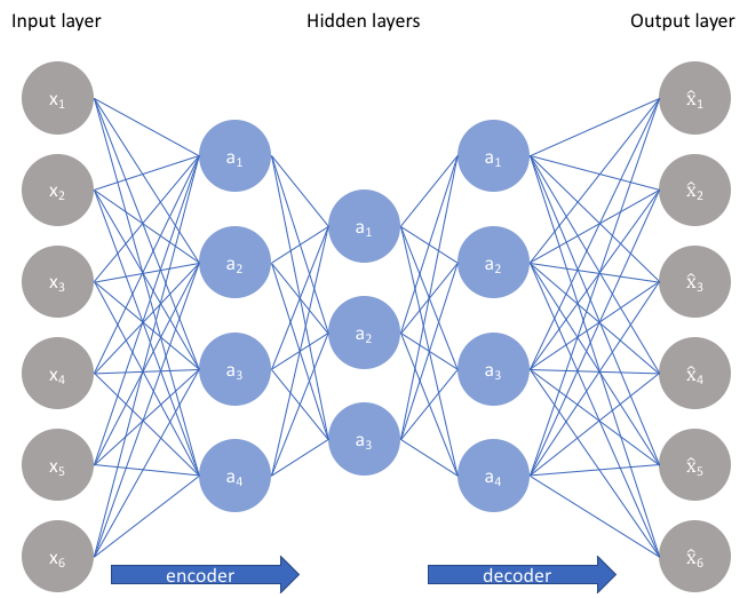
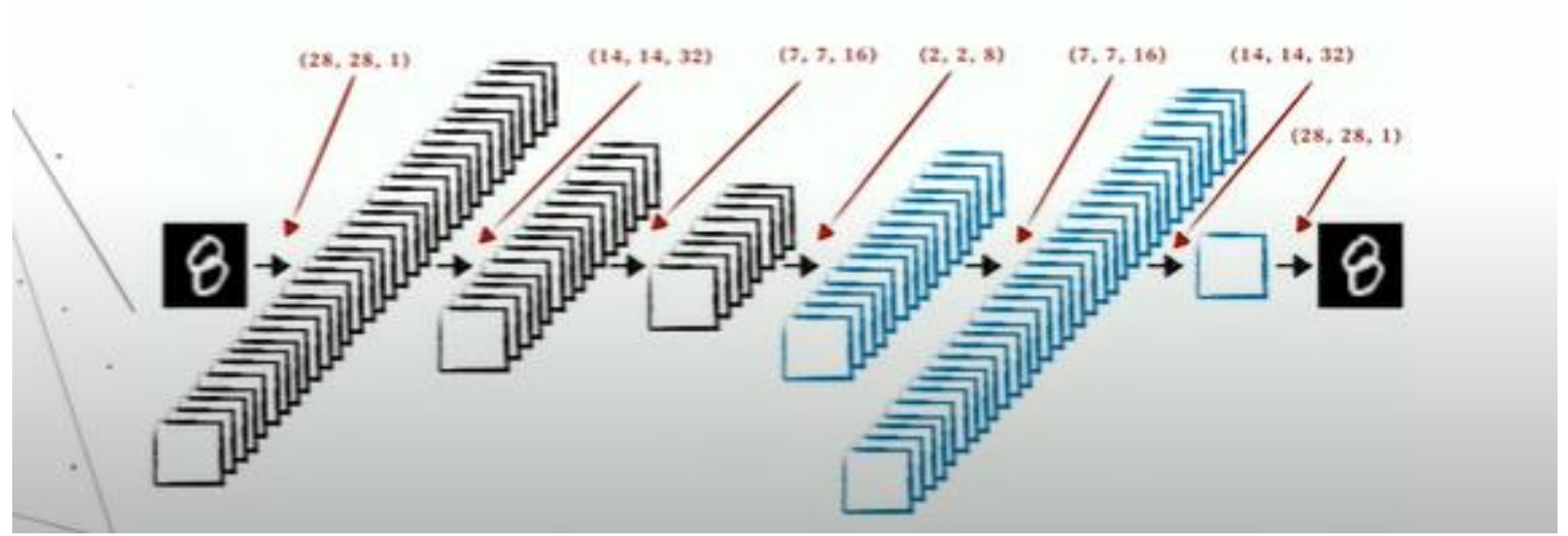
Firstly, we need to be made aware that an autoencoder has two operators, these being:

- **Encoder:** This transforms the input, x , into a low-dimensional latent vector, $z = f(x)$.
- Since the **latent vector is of low dimension**, the encoder is forced to learn only the most important features of the input data.
- **Decoder:** This tries to recover the input from the latent vector, $g(z) = \tilde{x}$.

Although the latent vector has a low dimension, it has a sufficient size to allow the decoder to recover the input data.



What encoder decoder is?



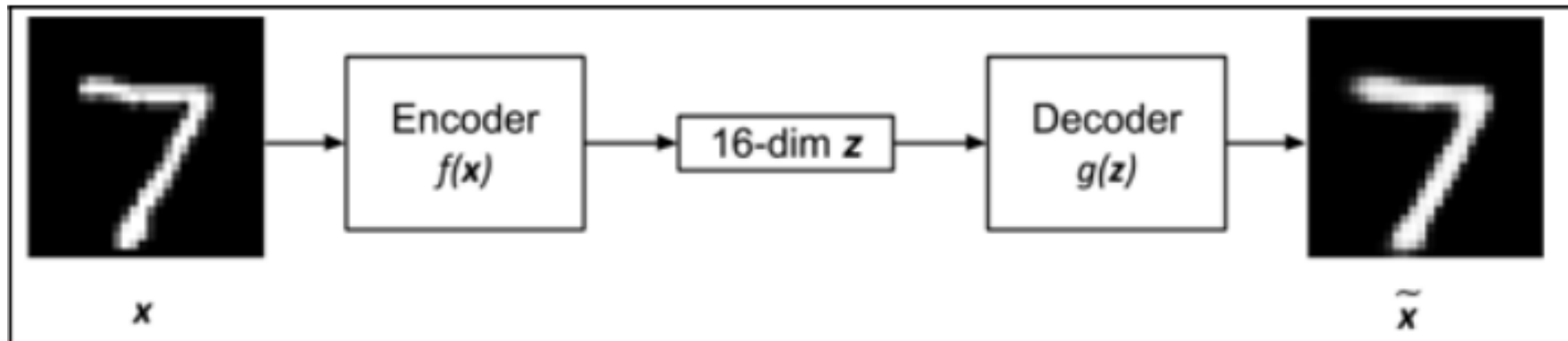
- The goal of the decoder is to make $\tilde{\mathbf{x}}$ as close as possible to $\mathbf{x}$.
- Generally, both the encoder and decoder are non-linear functions. (how?)
- The dimension of $\mathbf{z}$ is a measure of the number of important features it can represent.
- The dimension is usually much smaller than the input dimensions for efficiency and in order to constrain the latent code to learn only the most important properties of the input distribution .
- An auto encoder has the tendency to memorize the input when the dimension of the latent code is significantly bigger than $\mathbf{x}$.
- A suitable loss function, $\mathcal{L}(\mathbf{x}, \tilde{\mathbf{x}})$, is a measure of how dissimilar the input, $\mathbf{x}$, is from the output $\tilde{\mathbf{x}}$, which is the recovered input, $\mathbf{x}$. As shown in the following equation, the mean squared error (MSE) is an example of such a loss function:

$$\mathcal{L}(\mathbf{x}, \tilde{\mathbf{x}}) = MSE = \frac{1}{m} \sum_{i=1}^{i=m} (x_i - \tilde{x}_i)^2$$

- MNIST: Modified National Institute of Standards and Technology
- Hand written digits
- 60K Training and 10K Testing images
- Input: $28 \times 28 \text{px} = 784$, grayscale



- MNIST digit that has a dimension of $28 \times 28 \times 1 = 784$.
- The encoder transforms the input into a low-dimensional $\mathbf{z}$ that can be a 16-dimension latent vector.
- The decoder will attempt to recover the input in the form of $\tilde{x}$ from $\mathbf{z}$.



summary

Autoencoder Components:

Autoencoders consists of 4 main parts:

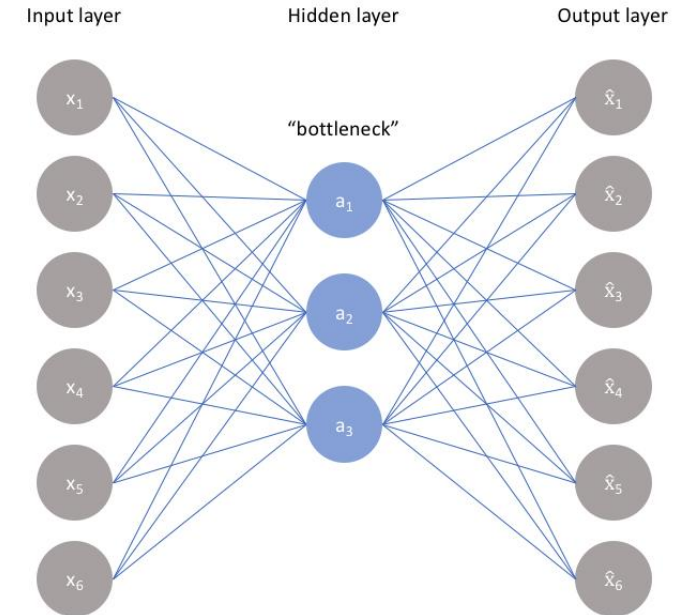
1- **Encoder**: In which the model learns how to reduce the input dimensions and compress the input data into an encoded representation.

2- **Bottleneck(latent vector)** : which is the layer that contains the compressed representation of the input data. This is the lowest possible dimensions of the input data.

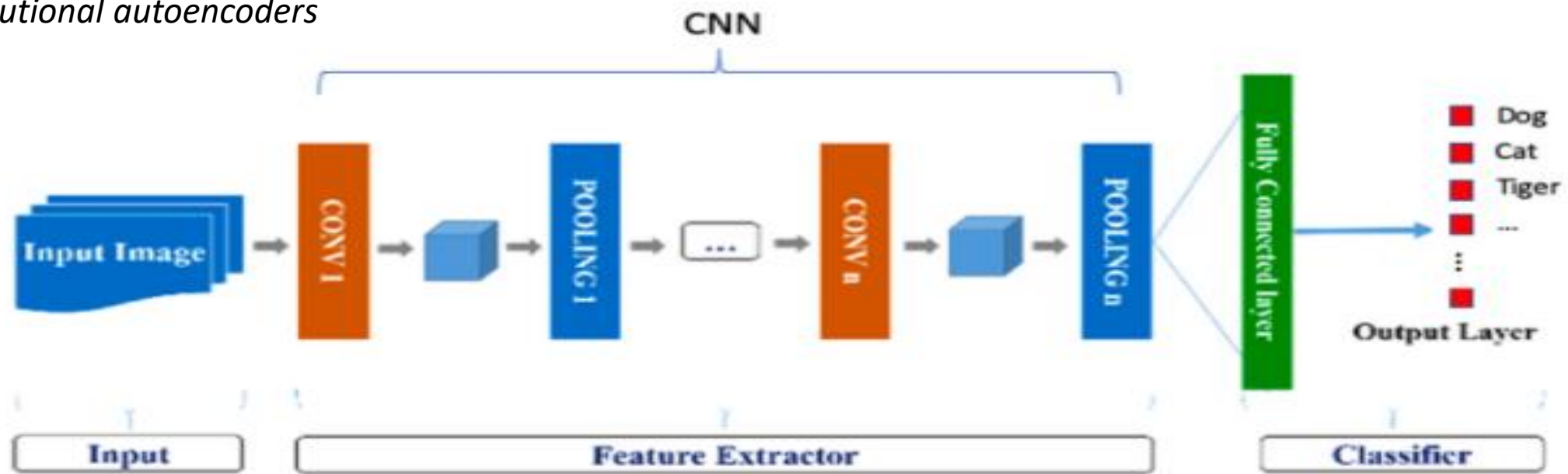
3- **Decoder**: In which the model learns how to reconstruct the data from the encoded representation to be as close to the original input as possible.

4- **Reconstruction Loss**: This is the method that measures measure how well the decoder is performing and how close the output is to the original input.

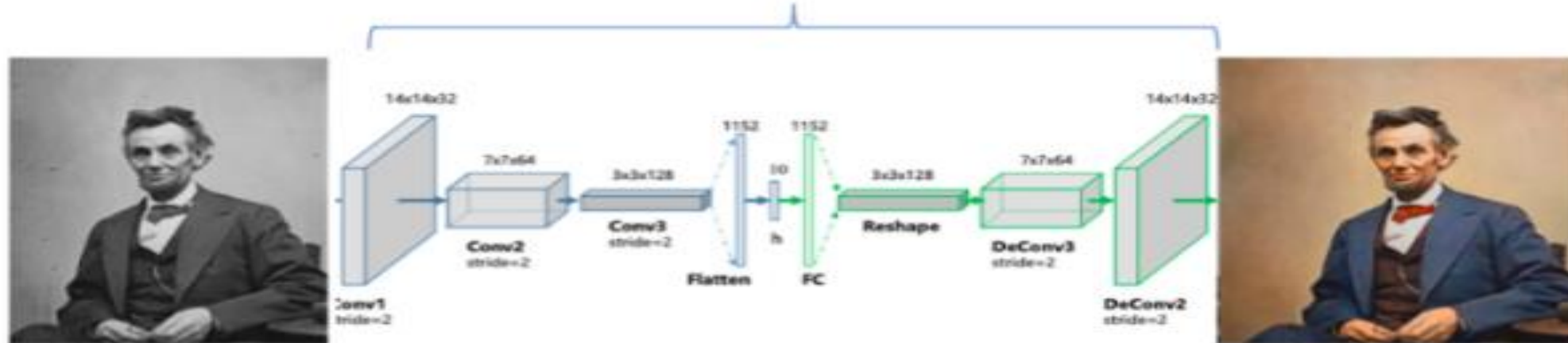
The training then involves using back propagation in order to minimize the network's reconstruction loss.



Convolutional autoencoders



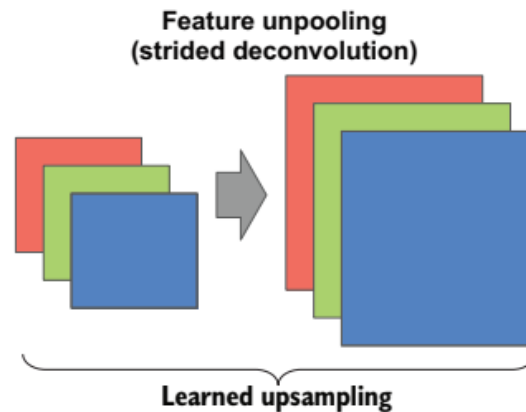
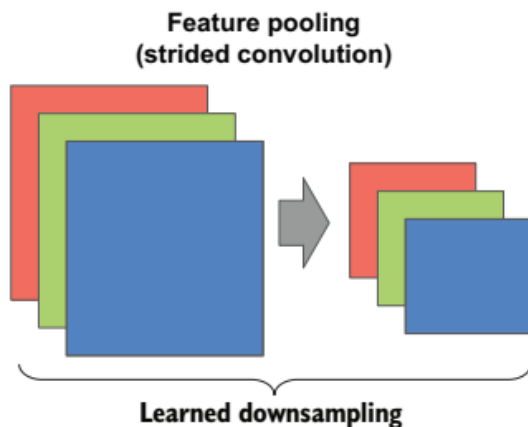
CNN Autoencoder



Convolutional autoencoders

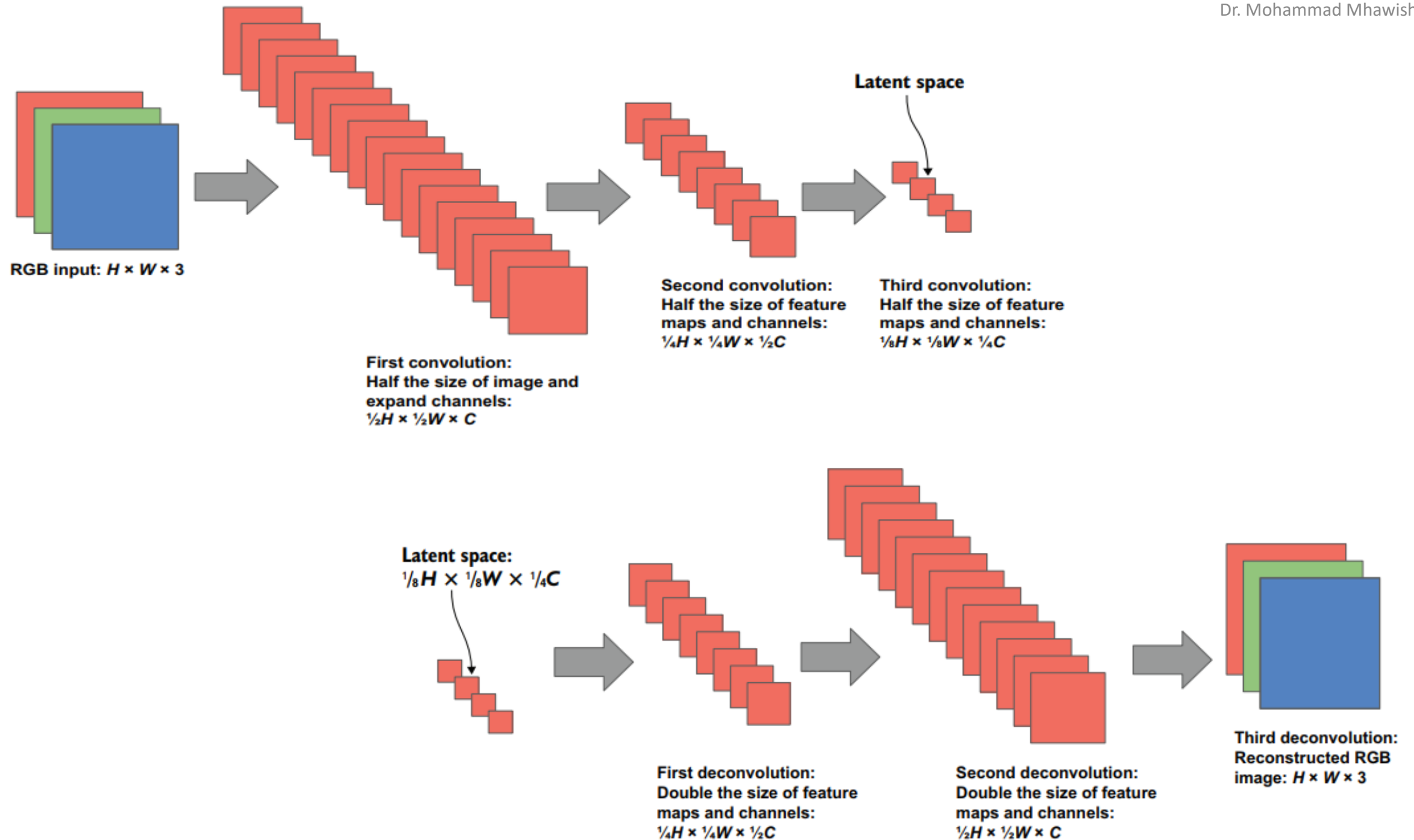
To do this, they use **convolutions** in the **encoder**, and **deconvolutions**(**transpose convolutions**) in the **decoder**.

- While a **strided convolution**, which does feature pooling, learns the optimal method to downsample a distribution,
- a **strided deconvolution** (feature unpooling) does the opposite and learns the optional method to upsample a distribution.



The **decoder** progressively **increases the number of feature maps** (via feature expansion) and the size of the feature maps (via feature unpooling) by using strided deconvolutions (transpose convolutions). The last unpooling layer projects the feature maps according to the reconstruction task

Transposed CNN (sometimes called deconvolution), which is more commonly used in decoders. We can imagine transposed CNN (**Conv2DTranspose**) as the reversed process of CNN.



Build the autoencoder

We are going to use the Functional API to build our convolutional autoencoder.

```
input = layers.Input(shape=(28, 28, 1))

# Encoder
x = layers.Conv2D(32, (3, 3), activation="relu", padding="same")(input)
x = layers.MaxPooling2D((2, 2), padding="same")(x)
x = layers.Conv2D(32, (3, 3), activation="relu", padding="same")(x)
x = layers.MaxPooling2D((2, 2), padding="same")(x)

# Decoder
x = layers.Conv2DTranspose(32, (3, 3), strides=2, activation="relu", padding="same")(x)
x = layers.Conv2DTranspose(32, (3, 3), strides=2, activation="relu", padding="same")(x)
x = layers.Conv2D(1, (3, 3), activation="sigmoid", padding="same")(x)

# Autoencoder
autoencoder = Model(input, x)
autoencoder.compile(optimizer="adam", loss="binary_crossentropy")
autoencoder.summary()
```

```
autoencoder.fit(
    x=train_data,
    y=train_data,
    epochs=50,
    batch_size=128,
    shuffle=True,
    validation_data=(test_data, test_data),
)
```

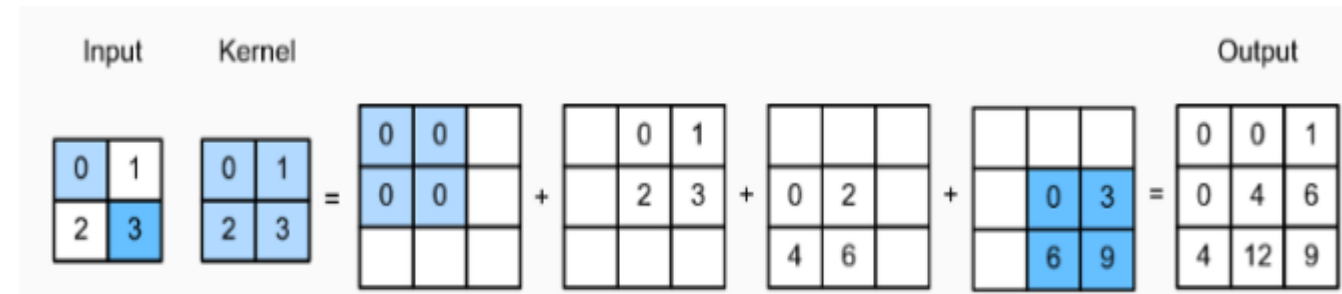
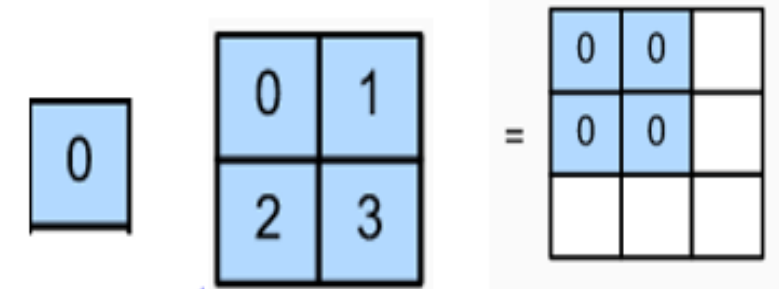
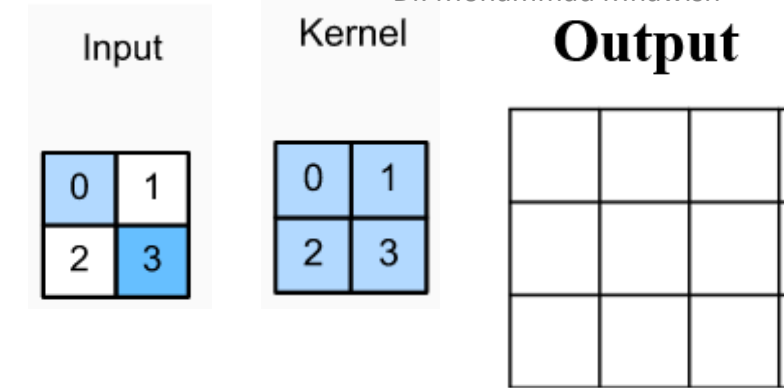
The tensor output of the encoder is also the input to a decoder that generates the output of the autoencoder.

In this example, we'll be using the crossentropy loss function and Adam optimizer.

During training, the input is the same as the output, x_train.

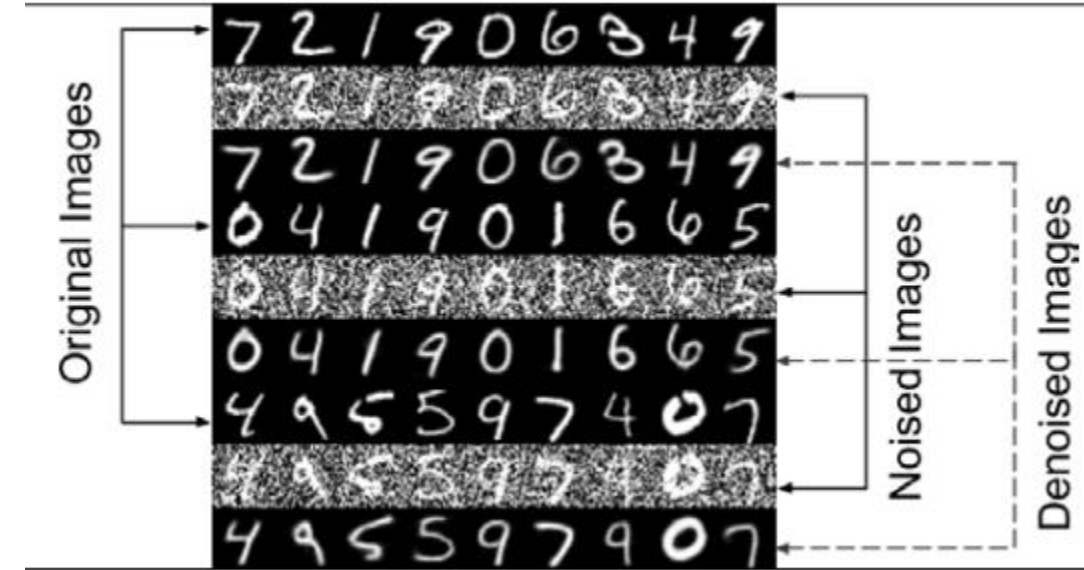
Transposed Convolutions

- Transposed Convolutions are used to **upsample** the input feature map to a desired output feature map using some learnable parameters. The basic operation that goes in a transposed convolution is below:
- 1. Consider a 2x2 encoded feature map which needs to be upsampled to a 3x3 feature map.
- 2. We take a kernel of size 2x2 with unit stride and zero padding.
- 3. Now we take the upper left element of the input feature map and multiply it with every element of the kernel
- 4. Similarly, we do it for all the remaining elements of the input feature map
- 5. As you can see, some of the elements of the resulting upsampled feature maps are over-lapping. To solve this issue, we simply add the elements of the over-lapping positions.
- 6. The resulting output will be the final upsampled feature map having the required spatial dimensions of 3x3.



Denoising autoencoders (DAEs)

- **Firstly,**
let's paint a picture and imagine that the MNIST digit images were corrupted by noise, thus making it harder for humans to read. We're able to build a denoising autoencoder (DAE) to remove the noise from these images..
- three sets of MNIST digits The top rows of each set (for example,
- MNIST digits 7, 2, 1, 9, 0, 6, 3, 4, and 9) are the original images.
- The middle rows show the inputs to the DAE, which are the original images corrupted by noise. As humans, we can find that it is difficult to read the corrupted MNIST digits.
- The last rows show the outputs of the DAE.



- the denoising autoencoder has practically the same structure as the autoencoder for MNIST that we presented in the previous section.
- The input is defined as: $\mathbf{x} = \mathbf{x}_{orig} + noise$



- In this formula, $\mathbf{x}_{-orig}$ represents the original MNIST image corrupted by *noise*.
- The objective of the encoder is to discover how to produce the latent vector, $\mathbf{z}$, which will enable the decoder to recover such as MSE, as shown through: $\mathbf{x}_{-orig}$ by minimizing the dissimilarity loss function:

$$\mathcal{L}(\mathbf{x}_{orig}, \tilde{\mathbf{x}}) = MSE = \frac{1}{m} \sum_{i=1}^{i=m} (x_{orig_i} - \tilde{x}_i)^2$$

This is like telling the autoencoder what the corrected images should be or asking it to figure out how to remove noise given a corrupted image.

```
autoencoder.fit(  
    x=noisy_train_data,  
    y=train_data,  
    epochs=100,  
    batch_size=128,  
    shuffle=True,  
    validation_data=(noisy_test_data, test_data),  
)
```

```
predictions = autoencoder.predict(noisy_test_data)  
display(noisy_test_data, predictions)
```



Automatic colorization autoencoder

another practical application of autoencoders. In this case, we're going to imagine that we have a grayscale photo and that we want to build a tool that will automatically add color to it. We would like to replicate the human abilities in identifying that the sea and sky are blue, the grass field and trees are green, while the clouds are white, and so on

Add some good noise
Called color



if we are given a grayscale photo (left) of a rice field in the foreground, a volcano in the background, and sky on the top, we're able to add the appropriate colors (right).

- A simple automatic colorization algorithm seems like a suitable problem for autoencoders. If we can train the autoencoder with a sufficient number of grayscale photos as input and the corresponding colored photos as output, it could possibly discover the hidden structure on properly applying colors. Roughly, it is the reverse process of denoising. The question is, can an autoencoder add color (good noise) to the original grayscale image?
- To keep things simple, we are going to use **CIFAR100 dataset**, which is readily available in Keras datasets. The dataset contains **50k colour** images of shape **32 * 32 * 3** for training, and **10k colour** images of the same shape for testing purpose.

```
from keras.datasets import cifar100
```
- we're able to use the `rgb2gray()` function to apply weights on R, G, and B components to convert from color to grayscale:

```
def rgb_2_gray(image):  
    return np.dot(image[..., :3], [0.299, 0.587, 0.114])
```

```

input_shape = (rows, cols, 1)
batch_size = 32
kernel_size = 3
latent_dim = 256
layer_filters = [64, 128, 256]

```

```

inputs = Input(shape = input_shape)
x = inputs
for filters in layer_filters:
    x = Conv2D(filters = filters,
              kernel_size = kernel_size,
              strides = 2,
              activation = 'relu',
              padding = 'same')(x)

```

```

shape = K.int_shape(x)
x = Flatten()(x)
latent = Dense(latent_dim, name = 'latent_vector')(x)
encoder = Model(inputs, latent, name = 'encoder')

```

```

latent_inputs = Input(shape = (latent_dim, ), name = 'decoder_input')
x = Dense(shape[1]*shape[2]*shape[3])(latent_inputs)
x = Reshape((shape[1], shape[2], shape[3]))(x)
# stack of Conv2DTranspose(256)-Conv2DTranspose(128)-
# Conv2DTranspose(64)
for filters in layer_filters[::-1]:
    x = Conv2DTranspose(filters = filters,
                      kernel_size = kernel_size,
                      strides = 2,
                      activation = 'relu',
                      padding = 'same')(x)
outputs = Conv2DTranspose(filters = channels,
                        kernel_size = kernel_size,
                        activation = 'sigmoid',
                        padding = 'same',
                        name = 'decoder_output')(x)
decoder = Model(latent_inputs, outputs, name = 'decoder')

```

```

autoencoder.fit(x_train_gray,
               x_train,
               validation_data = (x_test_gray, x_test),
               epochs = 30,
               batch_size = batch_size,
               callbacks = callbacks)

```

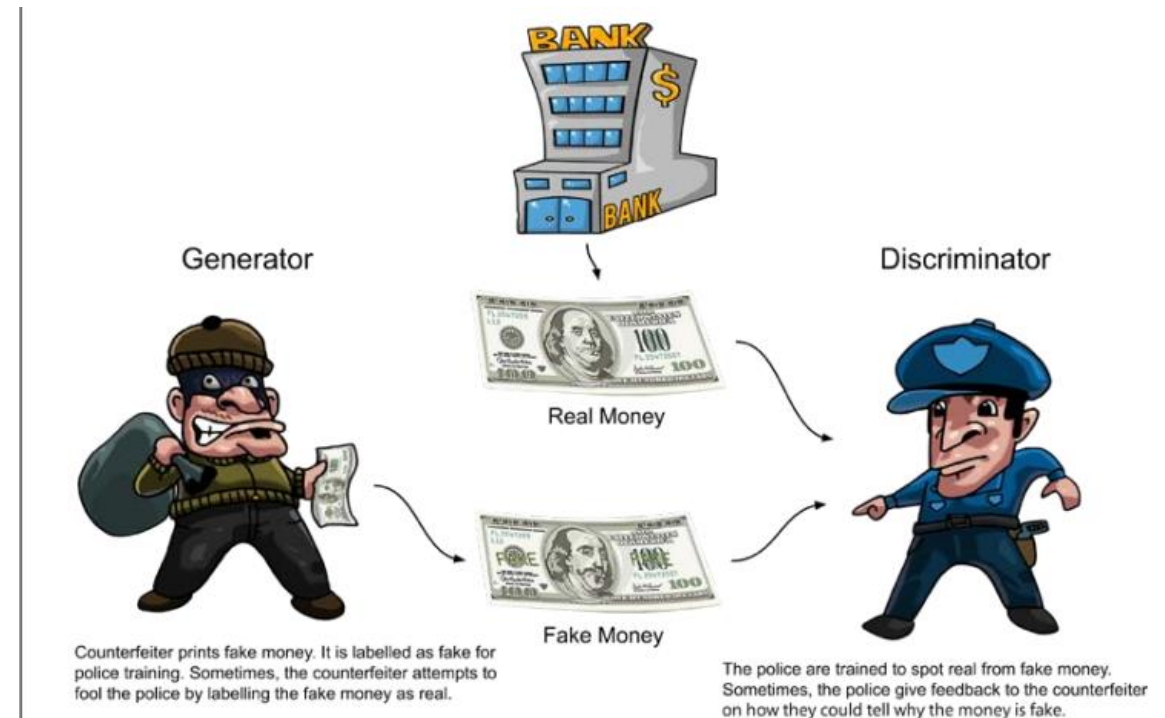
Generative adversarial networks (GANs)

- Generative adversarial networks (GANs) are a new type of neural architecture introduced by Ian Goodfellow and other researchers at the University of Montreal, including Yoshua Bengio, in 2014 .

- ## GAN architecture

GANs are based on the idea of *adversarial training*. The GAN architecture basically consists of two neural networks that compete against each other:

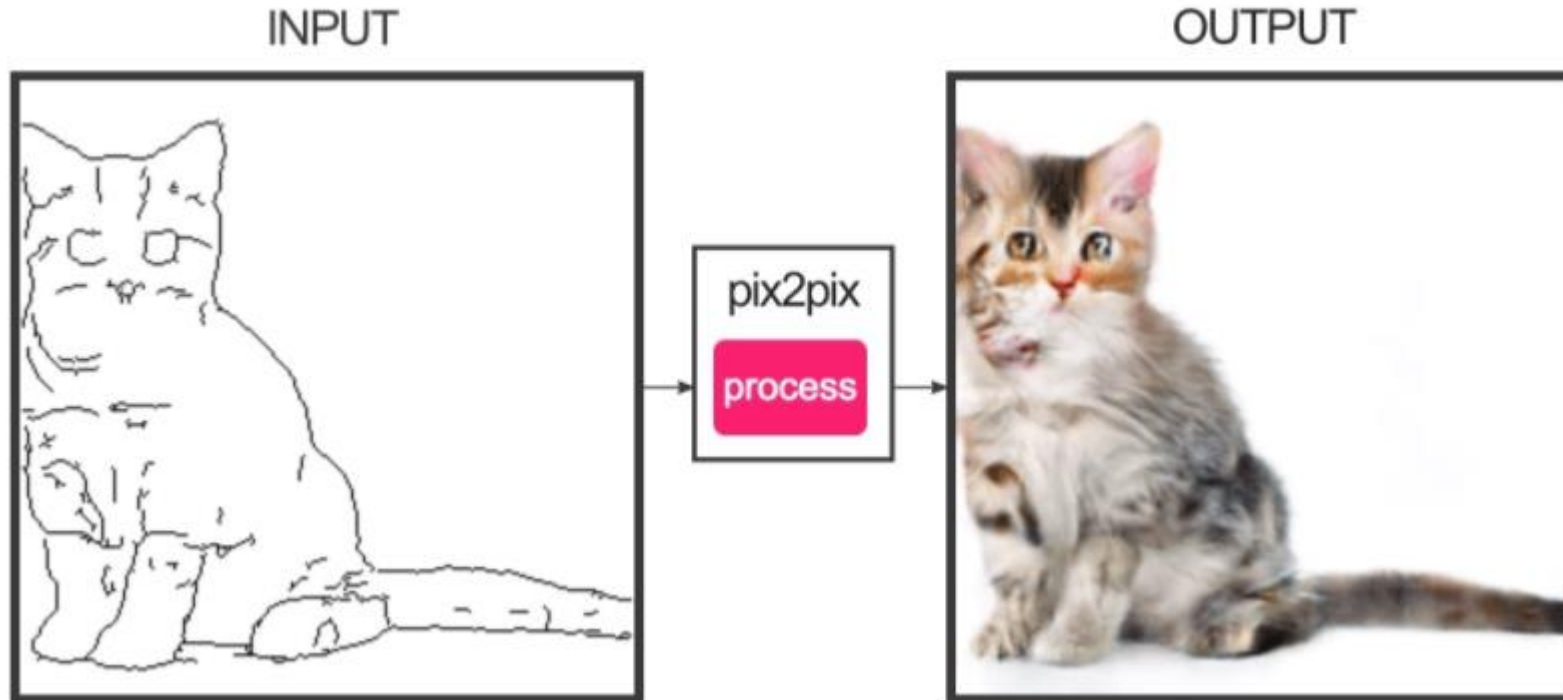
- The *generator* tries to convert random noise into observations that look as if they have been sampled from the original dataset.
- The *discriminator* tries to predict whether an observation comes from the original dataset or is one of the generator's forgeries.



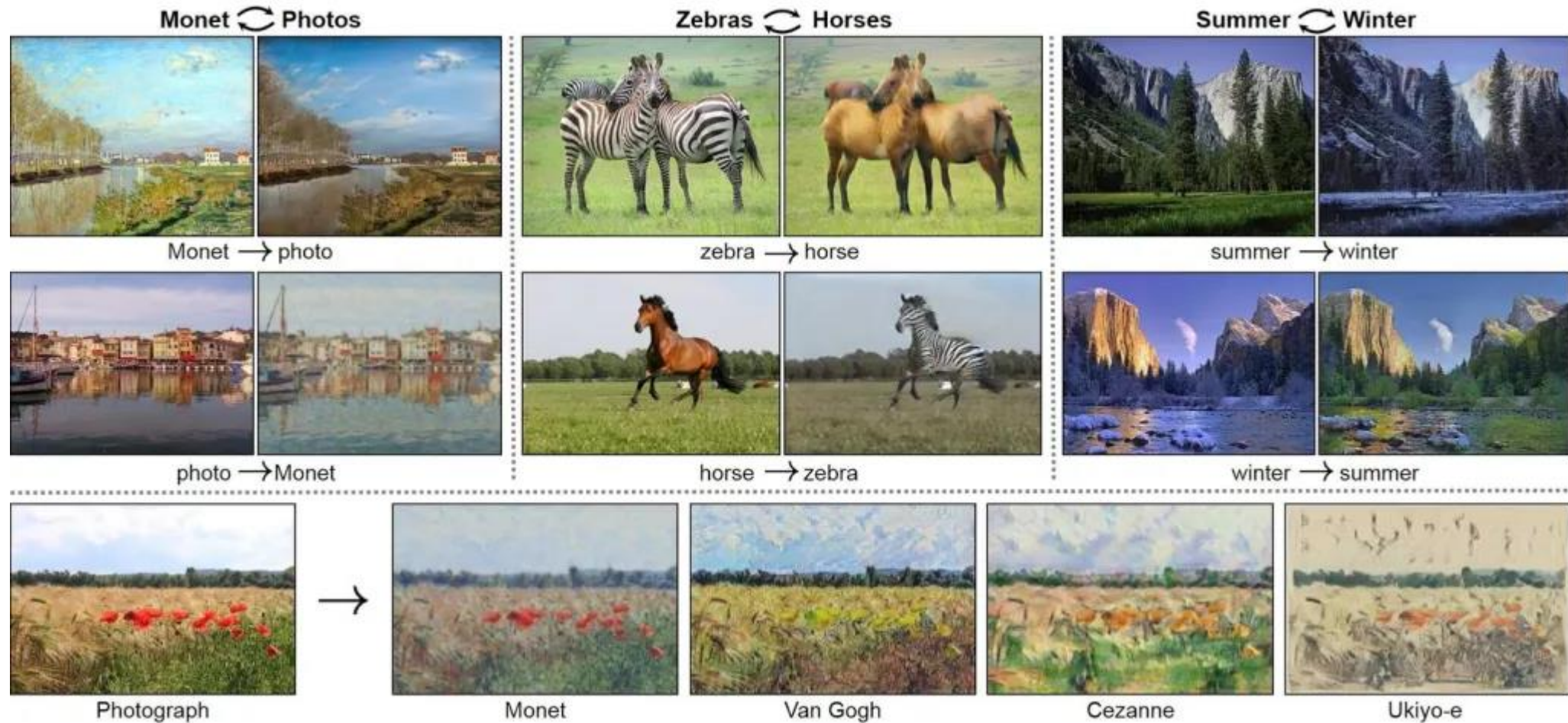
Application

Image-to-image translation

- Image-to-image translation is an application where a certain image B is transformed with the properties of A.



CycleGAN



Text-to-image synthesis

StackedGAN

- Stacked Generative Adversarial Networks (StackGAN) can generate images conditioned on text descriptions.

Text description	This bird is red and brown in color, with a stubby beak	The bird is short and stubby with yellow on its body	A bird with a medium orange bill white body gray wings and webbed feet	This small black bird has a short, slightly curved bill and long legs	A small bird with varying shades of brown with white under the eyes	A small yellow bird with a black crown and a short black pointed beak	This small bird has a white breast, light grey head, and black wings and tail
64x64 GAN-INT-CLS							
128x128 GAWWN							
256x256 StackGAN							

Face inpainting

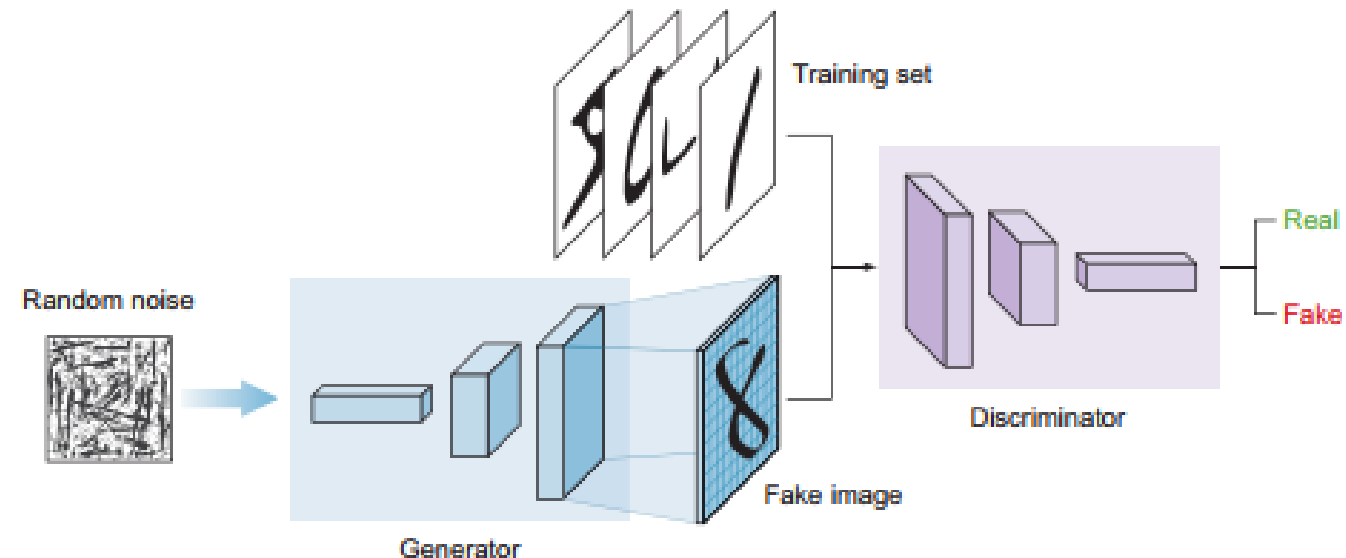
- Facial inpainting, also known as face completion, is the task of generating plausible facial features for missing pixels in a face image.



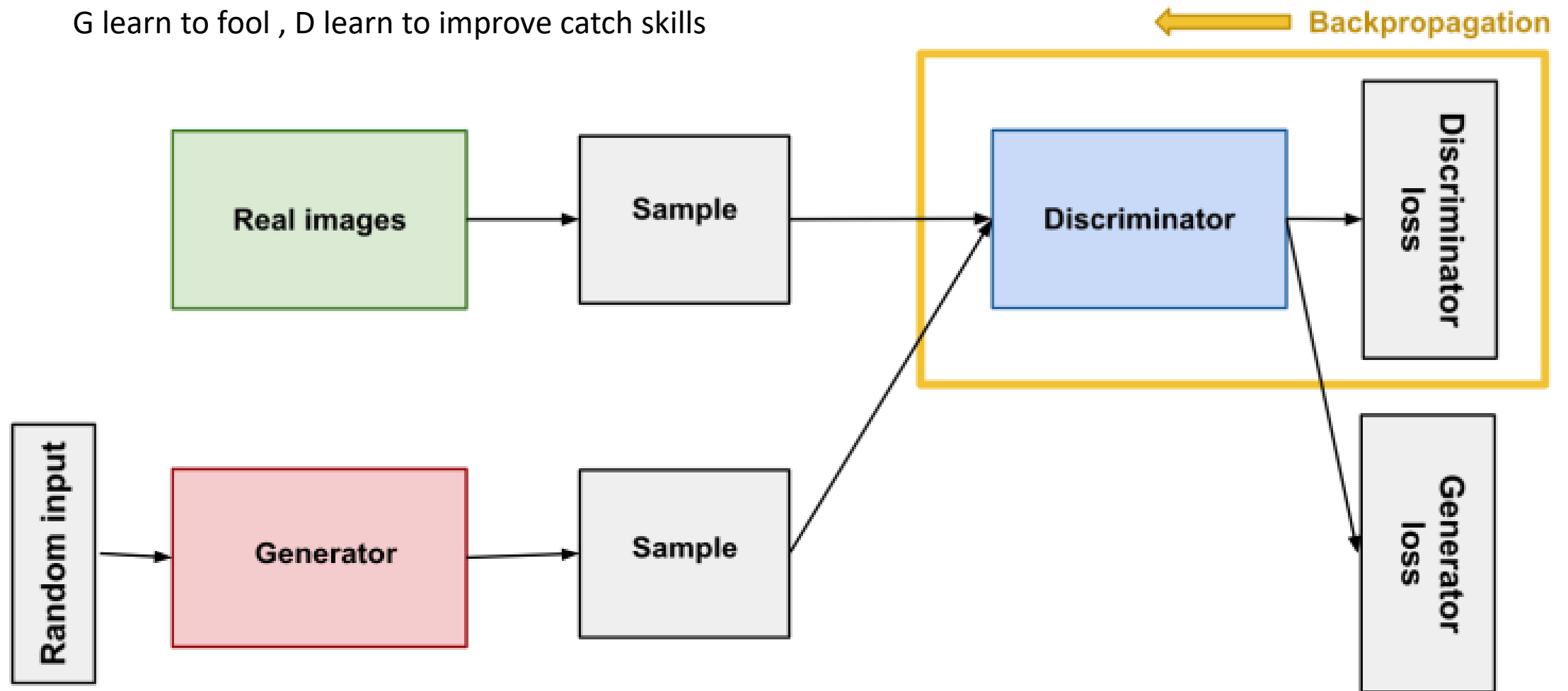
Principles of GANs

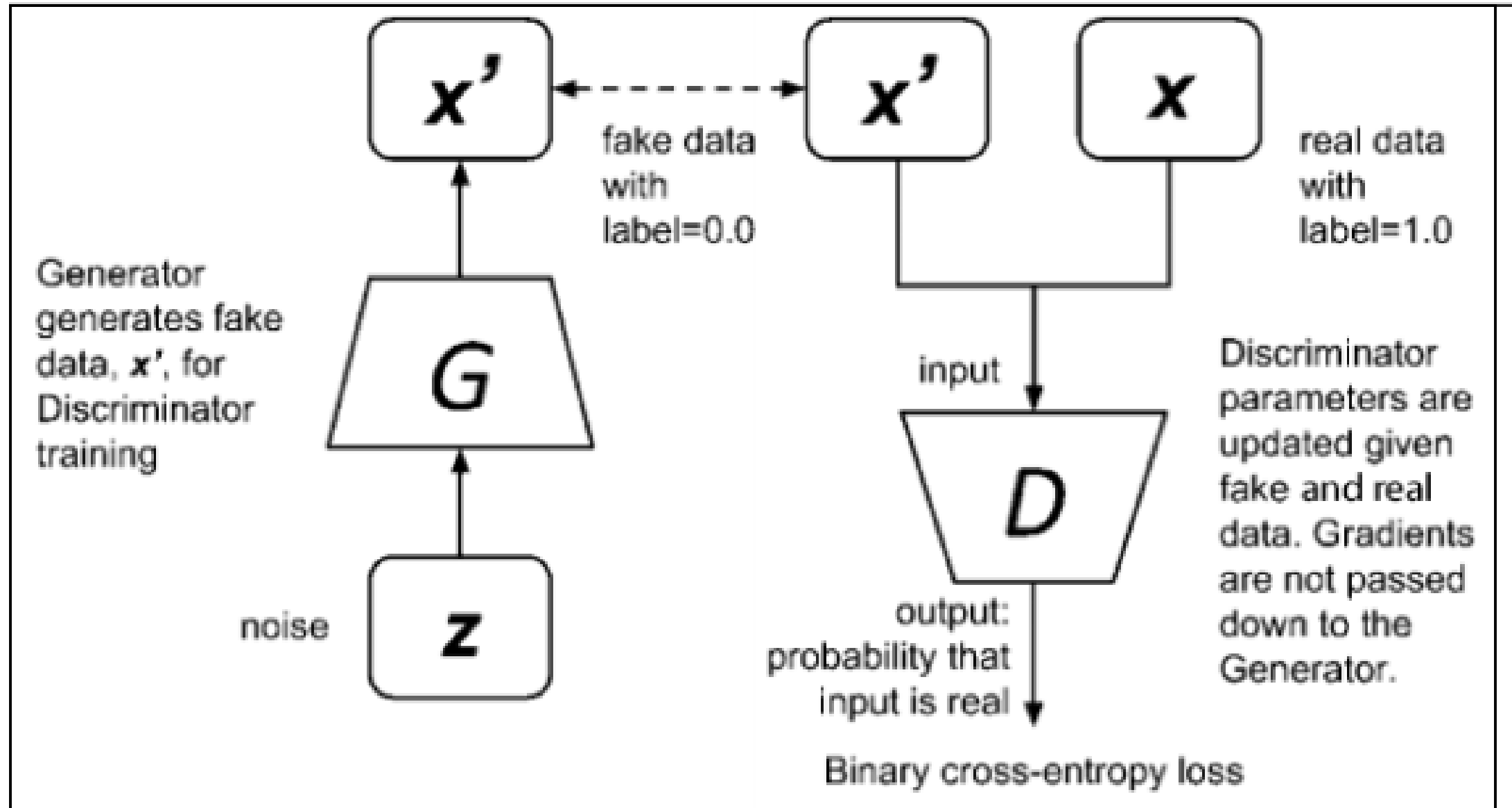
As you can see in the architecture diagram a GAN takes the following steps:

1. The generator takes in random numbers and returns an image.
2. This generated image is fed into the discriminator alongside a stream of images taken from the actual, ground-truth dataset.
3. The discriminator takes in both real and fake images and returns probabilities: numbers between 0 and 1, with 1 representing a prediction of authenticity and 0 representing a prediction of fake.



G learn to fool , D learn to improve catch skills





Researchers can make computers generate content of the same or higher quality compared to that created by their human counterparts.

By learning to mimic any distribution of data, computers can be taught to create worlds that are similar to our own in any domain: images, music, speech, prose.

Ex: <https://www.whichfaceisreal.com/index.php>

<https://generated.photos/face-generator/>

<https://thispersondoesnotexist.com/>